

Here's the 60-second summary of the paper:

The problem they identified

Standard MPPI computes a weighted average over all sampled trajectories.

Sometimes that average is nonsense.

Imagine:

- 100 good trajectories go left around an obstacle
- 100 good trajectories go right around an obstacle

The weighted average may go **straight through the obstacle**.

The paper shows a simple example where this happens.

2403.18066v1

Contribution #1: Rollout Clustering

Before averaging trajectories, they:

1. Sample trajectories normally.
2. Cluster them using DBSCAN.
3. Run the MPPI weighted average separately within each cluster.
4. Pick the best cluster.

Result:

- Left-going trajectories stay together.
- Right-going trajectories stay together.
- You don't average them into a collision.

This adds very little computational cost.

2403.18066v1

Contribution #2: Dynamic Obstacles

Normal MPPI assumes obstacles are static.

They instead:

1. Sample possible future trajectories of moving obstacles.
2. Add those predicted obstacle trajectories into the cost function.
3. Penalize trajectories likely to collide with where obstacles will be.

The key insight is they sample obstacle futures once and reuse them, avoiding a huge computational explosion.

2403.18066v1

Experiments

Not hardware.

They ran:

- 1000 randomly generated obstacle environments
- Dubins-car simulations
- Static obstacle tests
- Dynamic obstacle tests with 100 moving obstacles

all in MATLAB on a laptop.

2403.18066v1

Results

For static obstacles:

Method	Failures
Baseline MPPI	6.0%
Tube-MPPI	4.5%

Method	Failures
Clustered MPPI	2.9%

With control noise.

2403.18066v1

For dynamic obstacles:

Method	Collisions
MPPI	3
Clustered MPPI	2
Dynamic-Clustered MPPI	0

in their test scenario.

2403.18066v1

My assessment

The clustering idea is clever and practical.

The dynamic-obstacle idea is useful but feels more incremental.

What jumped out at me is that in their literature review they cite:

"Learning Sampling Distributions for Model Predictive Control" by Justin Sacks and Byron Boots (CoRL 2023)

2403.18066v1

which is almost exactly the direction you were proposing:

- learn where good trajectories tend to be,
- sample there instead of sampling blindly.

That citation might actually be the next paper you should read. It's closer to your "don't waste compute on obviously bad rollouts" intuition than the Bakolas paper itself.